CS Honors Project

Huffman Encoding Algorithm

CIS 22C: Data Abstractions and Structures

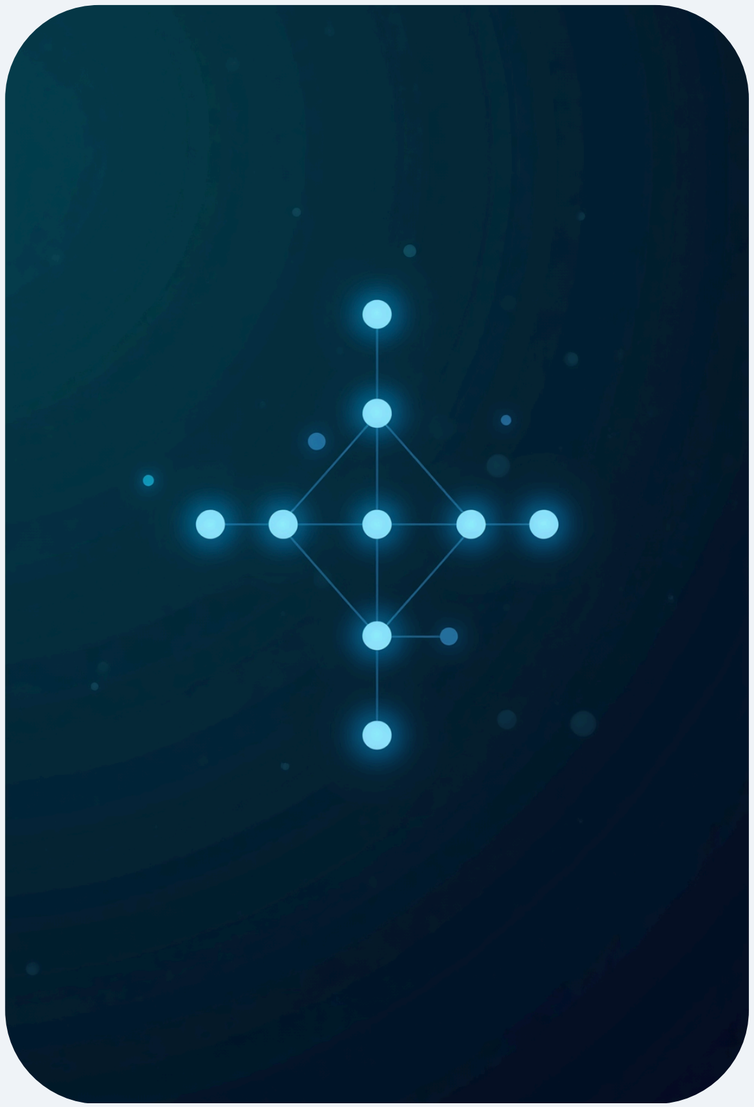
Group 2 — Aarush Muralinathan & Zhen Huey Lee

PRESENTED BY : AARUSH MURALINATHAN & ZHEN HUEY LEE

Introduction to Huffman Encoding

Huffman Encoding is a lossless data compression method that assigns variable-length binary codes to characters based on their frequency.

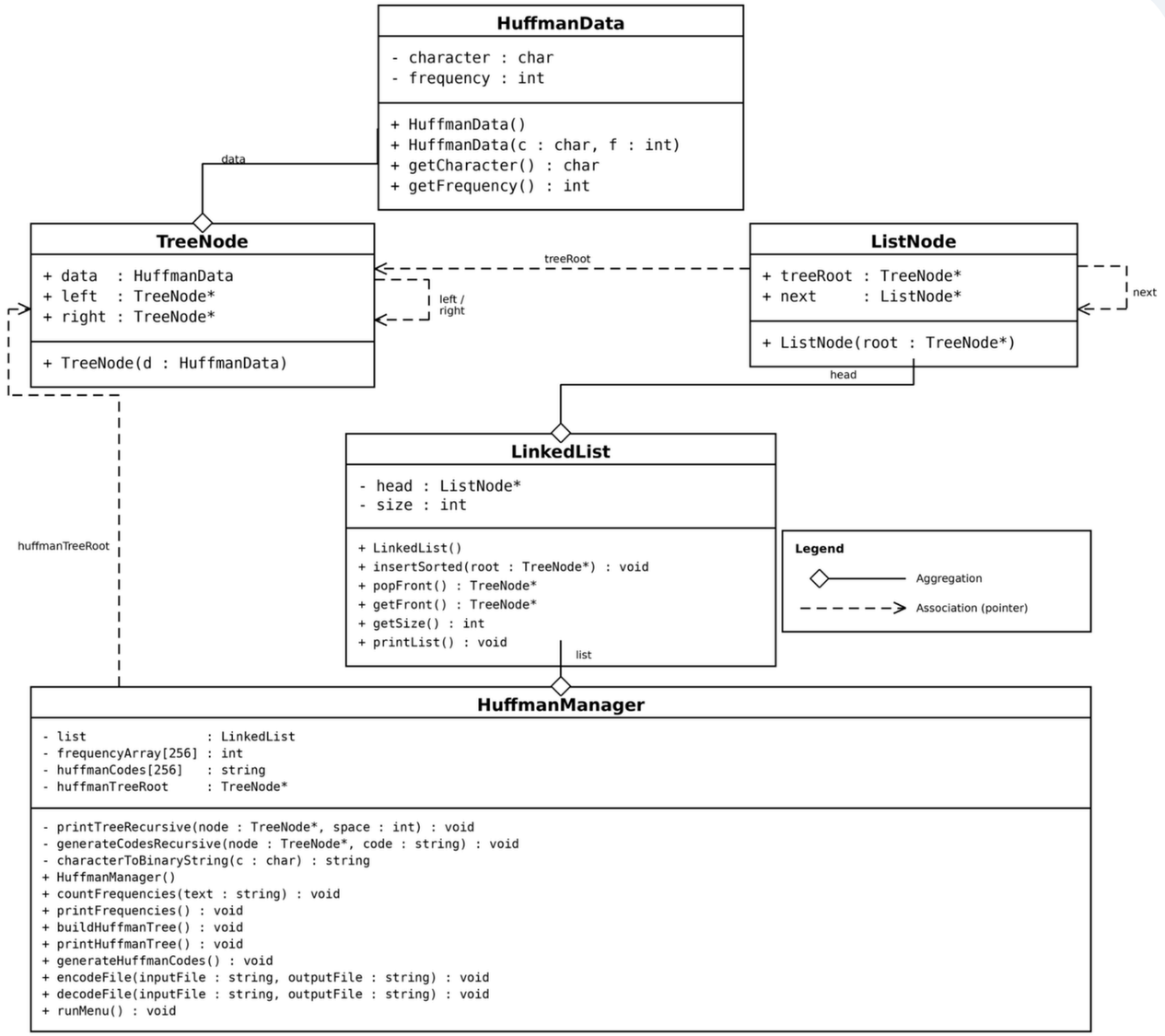
- Frequent characters receive shorter codes; rare characters receive longer codes
- Combines three core data structures: hash table (frequency array), singly linked list, and binary tree
- Menu-driven interface supports encoding, decoding, and file I/O operations
- Goal: minimize total bits used to represent a given input string



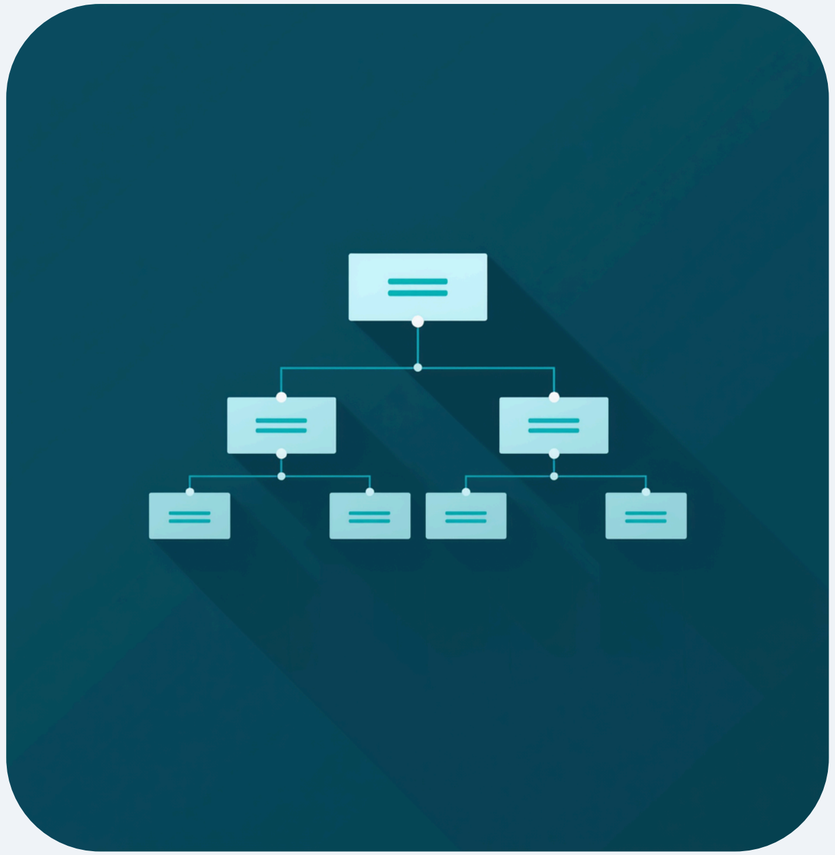
Data Structure Design Flow

Using "Mississippi" as the example input:
Frequency Array:
M = 1, p = 2, i = 4, s = 4
Sorted Linked List:
(M, 1) → (p, 2) → (i, 4) → (s, 4) → NULL
Huffman Tree (bottom-up merge):
Merge (M,1) + (p,2) → *(3)
Merge *(3) + (i,4) → *(7)
Merge *(7) + (s,4) → *(11) [root]
Result: frequent characters get shorter codes, rare characters get longer codes.

Class Diagram (UML)



Program Flow Structure Chart



main()
- HuffmanManager
- countFrequencies()
  - Reads input, builds frequency array / hash table
- buildTree()
  - Inserts nodes into sorted linked list
  - Constructs Huffman binary tree
- generateCodes()
  - Traverses tree, assigns binary codes
- runMenu() [Options 1–8]
- 1: Print character frequencies
- 2: Print Huffman tree
- 3: Print linked list
- 4: Print code table
- 5: Encode a word
- 6: Encode a file
- 7: Decode a file
- 8: Exit

Team Assignments & Collaboration

AARUSH MURALINATHAN

Data structure design, main driver program, menu interface, linked list insertion, file I/O operations, integration, and documentation.

ZHEN HUEY LEE

Binary tree logic and construction, recursive tree printing, and hash table frequency counting implementation.

MODULAR DESIGN

Modular design approach enabled independent testing of each component and ensured smooth integration across all modules.

COLLABORATION APPROACH

Tasks divided by core data structures: linked list and I/O vs. binary tree and hashing. Regular integration checkpoints ensured cohesive final output.

Sample Output Part 1

--- Huffman Encoding Menu ---

1. Print the character weights (frequencies)
2. Print the Huffman tree
3. Print the Huffman codes
4. Encode a word
5. Decode a bitstring
6. Encode a file
7. Decode a file
8. Exit

Enter your choice: 1

Character Frequencies:

'M' : 1 'i' : 4 'p' : 2 's' : 4

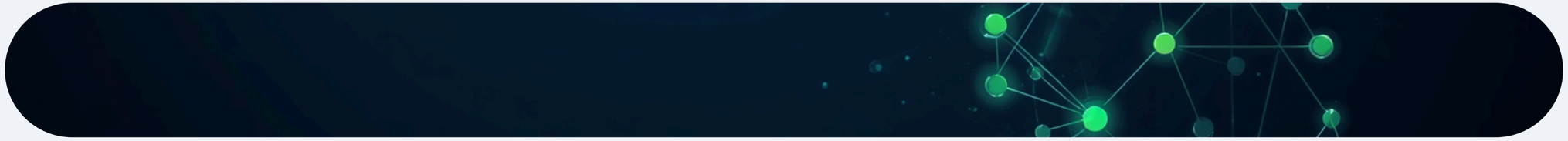
Linked List (Sorted): (M,1) -> (p,2) -> (i,4) -> (s,4) -> NULL

Sample Output Part 2

Option 2: Huffman Tree Traversal (Inorder Print)

Enter your choice: 2

```
        'i' (4)
    * (7)
            'p' (2)
        * (3)
            'M' (1)
* (11)
    's' (4)
```



Sample Output Part 3

Option 3:

Enter your choice: 3

Huffman Codes:

'M' : 100

'i' : 11

'p' : 101

's' : 0

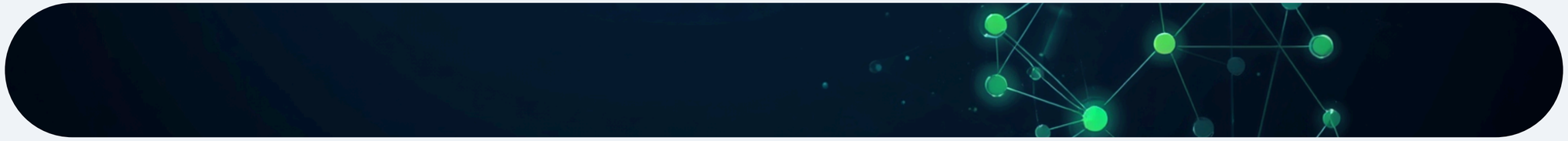
Sample Output Part 4

Option 4:

Enter your choice: 4

Enter a character: M

Huffman code for 'M': 100



Sample Output Part 5

Option 5:

Enter your choice: 5

Enter a word: Mis

ASCII Binary: 01001101 01101001 01110011

Huffman Binary: 100 11 0

Sample Output Part 6

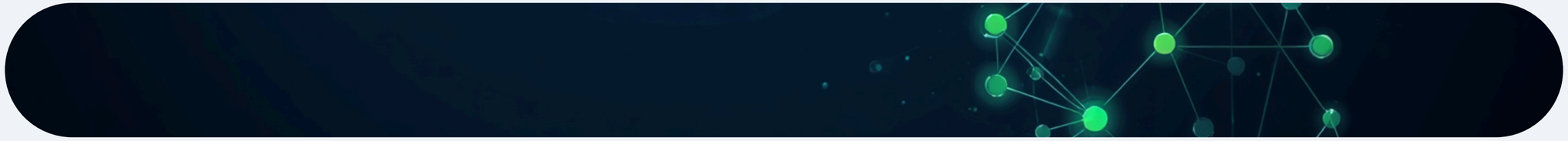
Option 6:

Enter your choice: 6

Enter input file name: in.txt

Enter output file name: encoded.txt

File encoded successfully to encoded.txt



Sample Output Part 7

Option 7:

Enter your choice: 7

Enter input file name: encoded.txt

Enter output file name: decoded.txt

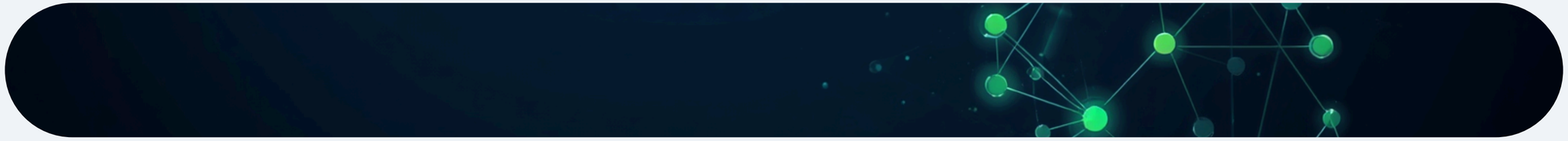
File decoded successfully to decoded.txt

Sample Output Part 8

Option 8:

Enter your choice: 8

Exiting program...



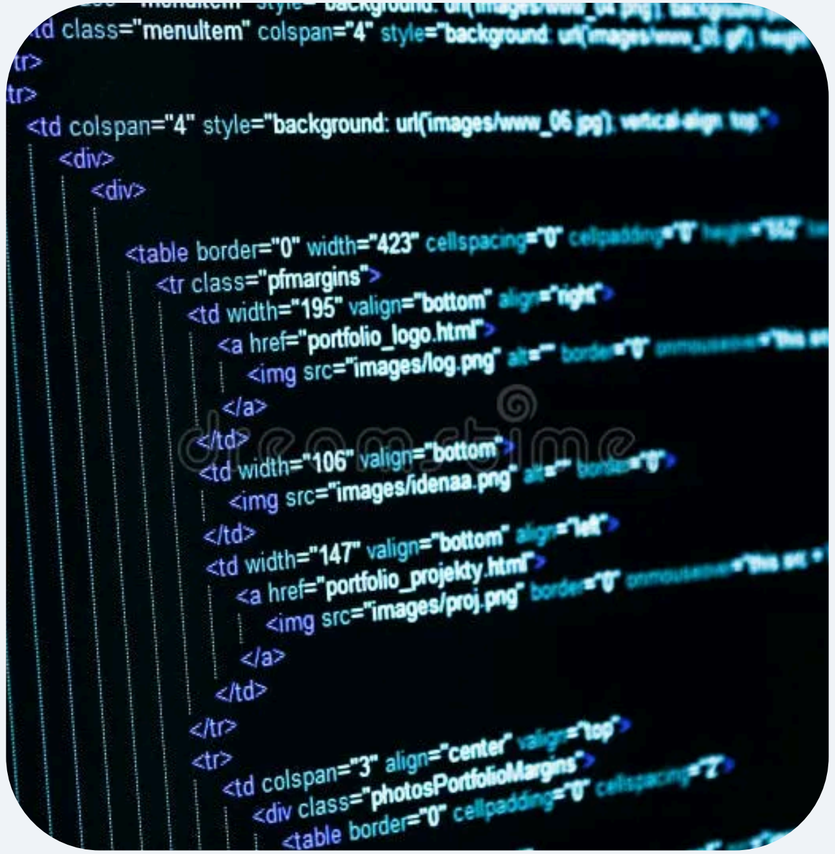
Major Debug Problems & Solutions

PROBLEM: NULL POINTER DEREFERENCING

Segmentation faults occurred during recursive tree traversal and linked list operations when nodes were accessed without checking for null pointers, causing the program to crash unexpectedly.

SOLUTIONS IMPLEMENTED

Added base case checks (if node != nullptr) in all recursive functions. Added safety checks in popFront() to handle empty linked list. Result: stable recursive traversals and safer list operations throughout the program.



Real-World Applications of Huffman Encoding

FILE COMPRESSION

Huffman encoding is widely used in file compression formats such as ZIP and GZIP, reducing file sizes by encoding frequent bytes with shorter bit sequences.

IMAGE COMPRESSION

Image formats like JPEG and PNG leverage Huffman coding as part of their compression pipeline to reduce image file sizes without significant quality loss.

AUDIO & VIDEO STREAMING

Audio and video codecs such as MP3 and MP4 use Huffman encoding to compress media data efficiently, enabling smooth streaming and reduced bandwidth usage.

WHY HUFFMAN ENCODING?

Huffman encoding is optimal for lossless compression — it guarantees the shortest average code length for a given set of symbol frequencies, making it a foundational algorithm in modern computing.

Thank You!

Successfully built a compression tool using arrays, linked lists, and binary trees.
Learned the importance of data structure choice and recursion with pointers.

GROUP 2: AARUSH MURALINATHAN & ZHEN HUEY LEE